Simulación del Problema Gravitacional de N Cuerpos

Camilo Huertas, Isabel Nieto

July 3, 2025

1 Introducción

Este documento describe la base teórica y la implementación de una simulación del movimiento de N cuerpos que interactúan mutuamente a través de la ley de la gravitación universal de Newton. El objetivo es simular este sistema sin realizar aproximaciones comunes, como suponer que una masa es dominante sobre las otras.

2 Fundamento Físico

Cada cuerpo de masa m_i, con un vector de posición $\vec{r}_i$, está sometido a la fuerza gravitacional ejercida por todos los demás cuerpos m_j en el sistema. La segunda ley de Newton para el cuerpo i se expresa como:

$$m_i \frac{d^2\vec{r}_i}{dt^2} = \vec{F}_i \tag{1}$$

donde $\vec{F}_i$ es la fuerza neta sobre el cuerpo i. Esta fuerza es la suma vectorial de las fuerzas gravitacionales individuales ejercidas por cada uno de los otros $N-1$ cuerpos:

$$\vec{F}_i = \sum_{j \neq i}^{N} \vec{F}_{ij} \tag{2}$$

La fuerza gravitacional $\vec{F}_{ij}$ ejercida por el cuerpo j sobre el cuerpo i está dada por la ley de la gravitación universal de Newton:

$$\vec{F}_{ij} = G \frac{m_i m_j}{|\vec{r}_j - \vec{r}_i|^3} (\vec{r}_j - \vec{r}_i) \tag{3}$$

donde G es la constante gravitacional universal. El vector $(\vec{r}_j - \vec{r}_i)$ apunta desde el cuerpo i hacia el cuerpo j, indicando que la fuerza es atractiva. La magnitud de la fuerza es inversamente proporcional al cuadrado de la distancia entre los cuerpos, $|\vec{r}_j - \vec{r}_i|^2$.

3 Unidades y Constantes

Para simplificar los cálculos numéricos, se utiliza un sistema de unidades normalizadas. En este sistema, la constante gravitacional G se establece en 1:

$$G = 1.0 \tag{4}$$

Esto permite trabajar con masas, distancias y tiempos en términos relativos, sin necesidad de manejar los órdenes de magnitud extremadamente grandes o pequeños que aparecen al usar unidades estándar del SI.

4 Integración Numérica: Algoritmo de Verlet

Para resolver el sistema de ecuaciones diferenciales de segundo orden (Ecuación 3 combinada con la segunda ley de Newton), se emplea un método de integración numérica. El algoritmo de Verlet es una elección popular para simulaciones de dinámica molecular y sistemas N-cuerpos debido a su buena estabilidad a largo plazo y conservación de energía, además de ser reversible en el tiempo.

La forma del algoritmo de Verlet utilizada es la conocida como "velocity Verlet":

$$\vec{r}_i(t + \Delta t) = \vec{r}_i(t) + \vec{v}_i(t)\Delta t + \frac{1}{2}\vec{a}_i(t)(\Delta t)^2 \tag{5}$$

$$\vec{v}_i(t + \Delta t) = \vec{v}_i(t) + \frac{1}{2}\left(\vec{a}_i(t) + \vec{a}_i(t + \Delta t)\right)\Delta t \tag{6}$$

donde:

- $\vec{r}_i(t)$, $\vec{v}_i(t)$, y $\vec{a}_i(t)$ son la posición, velocidad y aceleración del cuerpo i en el tiempo t, respectivamente.
- Δt es el paso de tiempo de la integración.
- $\vec{a}_i(t) = \vec{F}_i(t)/m_i$.
- $\vec{a}_i(t+\Delta t)$ es la aceleración del cuerpo i en el tiempo $t+\Delta t$, calculada usando las nuevas posiciones $\vec{r}_k(t + \Delta t)$ de todos los cuerpos k obtenidas de la Ecuación 5.

El procedimiento en cada paso de tiempo Δt es:

1. Calcular $\vec{F}_i(t)$ para cada cuerpo i basándose en las posiciones $\vec{r}_k(t)$.
2. Calcular $\vec{a}_i(t) = \vec{F}_i(t)/m_i$.
3. Actualizar las posiciones a $\vec{r}_i(t + \Delta t)$ usando la Ecuación 5.
4. Calcular las nuevas fuerzas $\vec{F}_i(t + \Delta t)$ usando las nuevas posiciones $\vec{r}_k(t + \Delta t)$.
5. Calcular las nuevas aceleraciones $\vec{a}_i(t + \Delta t) = \vec{F}_i(t + \Delta t)/m_i$.
6. Actualizar las velocidades a $\vec{v}_i(t + \Delta t)$ usando la Ecuación 6.

5 Conservación de la Energía

En un sistema gravitacional aislado, la energía mecánica total E debe conservarse. Esta energía es la suma de la energía cinética total K y la energía potencial total U:

$$E = K + U \tag{7}$$

La energía cinética total es:

$$K = \sum_{i=1}^{N} \frac{1}{2} m_i |\vec{v}_i|^2 \tag{8}$$

La energía potencial gravitacional total es la suma de las energías potenciales de todos los pares de cuerpos únicos:

$$U = -\sum_{i=1}^{N} \sum_{j=i+1}^{N} G \frac{m_i m_j}{|\vec{r}_i - \vec{r}_j|} \tag{9}$$

El seguimiento de la energía total del sistema a lo largo de la simulación es una forma importante de verificar la precisión y estabilidad del integrador numérico.

6 Implementación del Software

El software se desarrolla en C++ utilizando un enfoque de Programación Orientada a Objetos (POO).

- **Clase 'vector3D'**: Proporciona funcionalidades para álgebra vectorial en 3 dimensiones (suma, resta, producto escalar, producto vectorial, norma, etc.). Esta clase es fundamental para manejar las cantidades vectoriales como posición, velocidad y fuerza.
- **Clase 'Cuerpo'**: Encapsula las propiedades de cada cuerpo celeste (masa m, radio R, posición $\vec{r}$, velocidad $\vec{V}$, y fuerza $\vec{F}$). Incluye métodos para inicializar el cuerpo, calcular la fuerza gravitacional debida a otro cuerpo, y actualizar su estado (posición y velocidad) mediante el algoritmo de Verlet.

- **Lógica Principal (‘main.cpp‘)**: Gestiona la simulación completa, incluyendo:
  - Solicitud de datos iniciales al usuario (número de cuerpos, sus propiedades, parámetros de simulación como Δt y tiempo total).
  - Verificación de la validez de los datos de entrada.
  - El bucle principal de simulación que itera en el tiempo, aplicando el algoritmo de Verlet.
  - Cálculo y almacenamiento de las fuerzas, posiciones, velocidades y energías en cada paso.
  - Salida de resultados a un archivo de datos.
  - Interfaz para la visualización de resultados mediante herramientas externas (Gnuplot, Python/Matplotlib, Octave).

La estructura del proyecto está organizada en directorios para ‘include‘ (archivos de cabecera), ‘src‘ (archivos fuente), ‘bin‘ (ejecutable), ‘results‘ (datos de salida), ‘scripts‘ (scripts de graficación) y ‘documents‘ (documentación). Un ‘Makefile‘ gestiona la compilación del proyecto, la generación de documentación (Doxygen) y la creación de este documento PDF (LaTeX).